

Descrição OAT - API de Streaming

Contexto

Você foi contratado para desenvolver o backend de uma plataforma de streaming de vídeos semelhante à Netflix.

A aplicação deve permitir:

- Cadastro de usuários
- Cadastro de vídeos
- Histórico de visualização
- Sistema de avaliação
- Ranking de conteúdos mais assistidos

O foco do projeto é a **modelagem de dados em MongoDB**, boas práticas de consulta e uso de aggregation.

Objetivo da Atividade

Desenvolver uma API em **Node.js + Express + MongoDB (Mongoose)** capaz de:

1. Criar e listar usuários
2. Criar e listar vídeos
3. Registrar histórico de visualização
4. Calcular estatísticas usando aggregation
5. Implementar indexação adequada

Requisitos Funcionais

Usuários

Cada usuário deve possuir:

- Nome
- Email
- Data de criação
- Lista de perfis (ex: infantil, adulto)

Exemplo esperado:

```
{
  "name": "João Silva",
  "email": "joao@email.com",
  "profiles": [
    { "name": "João Kids", "ageGroup": "infantil" },
    { "name": "João Adulto", "ageGroup": "adulto" }
  ],
  "createdAt": "2026-03-01"
}
```

Vídeos

Cada vídeo deve possuir:

- Título
- Categoria
- Duração (em minutos)
- Número de visualizações
- Nota média
- Data de cadastro

Histórico de Visualização

Cada registro deve conter:

- ID do usuário
- ID do vídeo
- Data de visualização
- Progresso (em %)
- Avaliação (opcional, de 1 a 5)

Regras de Negócio

1. Ao registrar uma visualização:

- O contador de visualizações do vídeo deve ser incrementado.

1. Caso o usuário avalie o vídeo:

- A nota média deve ser recalculada.

1. O sistema deve permitir:

- Listar os 10 vídeos mais assistidos.
- Listar vídeos por categoria.
- Buscar histórico de um usuário.

Requisitos Técnicos Obrigatórios

O projeto deve conter:

- Uso de **Mongoose**
- Uso de pelo menos:
 - \$inc
 - \$group
 - \$sort
 - \$lookup
- Pelo menos 2 índices definidos manualmente
- Uso de variáveis de ambiente para conexão com MongoDB
- Estrutura organizada de pastas



Desafio Extra (Opcional)

1. Criar recomendação simples baseada em categoria mais assistida.
2. Criar TTL index para apagar históricos com mais de 1 ano.
3. Implementar paginação nos endpoints.